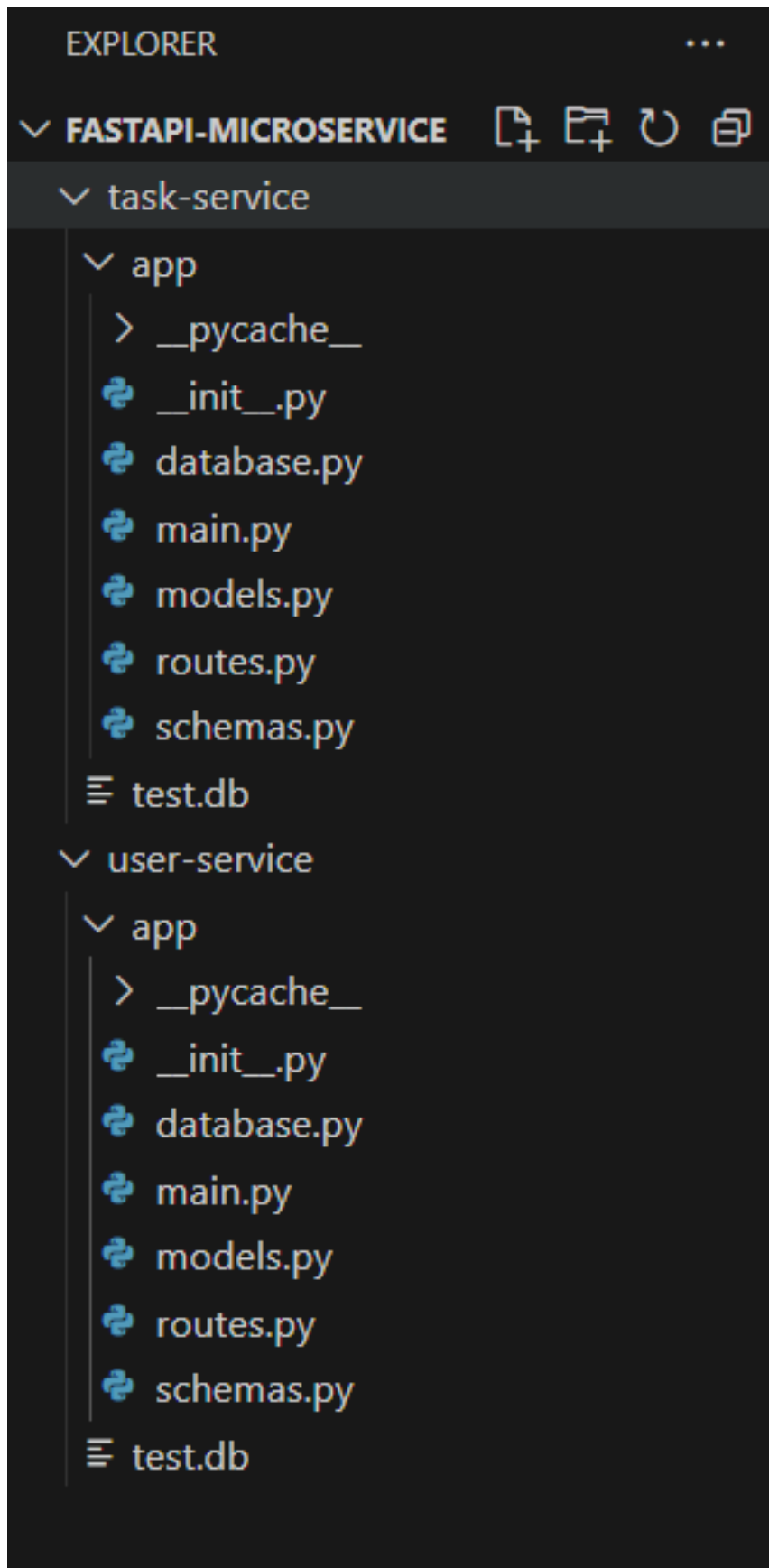


Ejercicio 2

Implementación de la creación de microservicios con FastAPI

Paso 1: Crear los microservicios

Cree los microservicios de Usuario y Tarea utilizando FastAPI. Una vez creados los microservicios, la estructura de archivos se verá como en la siguiente imagen.



Paso 2: Implementación del servicio de usuario

database.py : Crea la clase de base de datos y se guardará como **user-service/app/database.py** y coloca el siguiente código.

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# SQLite database URL
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

# Create the database engine
engine = create_engine(SQLALCHEMY_DATABASE_URL)
# Create a configured "Session" class
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
# Create a base class for declarative class definitions
Base = declarative_base()

# Dependency to get the database session

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

models.py : Crea la clase models y se guardará como **user-service/app/models.py** y coloca el siguiente código.

```
from sqlalchemy import Column, Integer, String
from .database import Base

# Define the Task model
class Task(Base):
    __tablename__ = "tasks"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    description = Column(String)
```

routes.py: Crea la clase routes y se guardará como **user-service/app/routes.py** y coloca el siguiente código.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from . import models, schemas
from .database import get_db

# Initialize the router
router = APIRouter()

# Create a new task endpoint
```

```

@router.post("/", response_model=schemas.Task)
def create_task(task: schemas.TaskCreate, db: Session = Depends(get_db)):
    db_task = models.Task(title=task.title,
description=task.description)
    db.add(db_task)
    db.commit()
    db.refresh(db_task)
    return db_task

# Get a task by ID endpoint
@router.get("/{task_id}", response_model=schemas.Task)
def read_task(task_id: int, db: Session = Depends(get_db)):
    db_task = db.query(models.Task).filter(models.Task.id == task_id).first()
    if db_task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return db_task

```

schemas.py : Crea la clase schemas y se guardará como **user-service/app/schemas.py** y coloca el siguiente código.

```

from pydantic import BaseModel

# Define the TaskCreate schema
class TaskCreate(BaseModel):
    title: str
    description: str

# Define the Task schema
class Task(BaseModel):
    id: int
    title: str
    description: str

class Config:
    orm_mode = True # Enable ORM mode to work with SQLAlchemy models

```

main.py : Crea la clase principal y se guardará como **user-service/app/main.py** y coloca el siguiente código.

```

from fastapi import FastAPI
desde . motor de importación de base de datos
desde . modelos importar Base
from .routes import router as task_router

# Crear todas las tablas de la base de datos
Base.metadata.create_all ( bind = engine )

# Inicializar la aplicación FastAPI

```

```
aplicación = FastAPI ()
```

Incluir el enrutador de tareas con el prefijo y las etiquetas especificadas

```
app.include_router ( task_router , prefix = "/tasks" , tags = [ " tasks" ] )
```

```
@app . obtener ( "/" )
```

```
def read_root ():
```

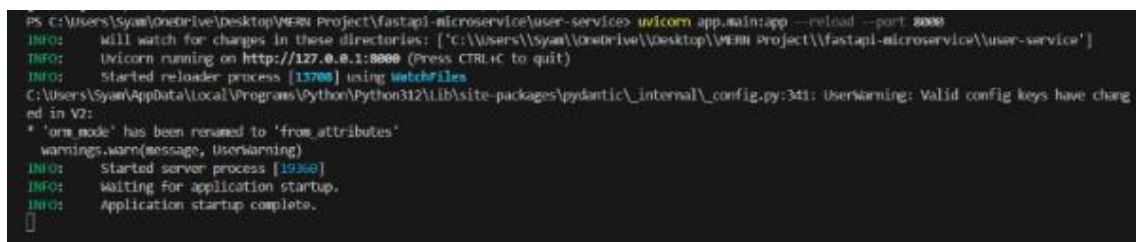
```
    return { "mensaje" : "Bienvenido al Servicio de Tareas" }
```

Paso 3: Ejecutar el servicio de usuario

Podemos usar el siguiente comando para ejecutar el servicio de usuario.

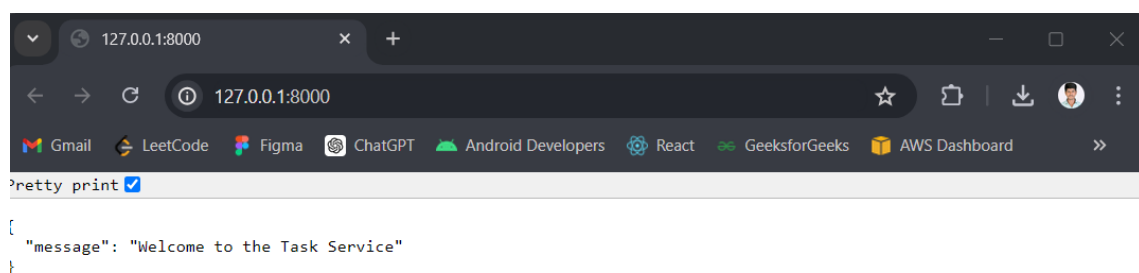
```
uvicorn app.main:app --reload --port 8000
```

Consulte la siguiente imagen para ver el proceso en ejecución.



```
PS C:\Users\Syam\OneDrive\Desktop\WEBN Project\Fastapi-microservice\user-service> uvicorn app.main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\Users\Syam\OneDrive\Desktop\WEBN Project\Fastapi-microservice\user-service']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15788] using WatchFiles
C:\Users\Syam\AppData\Local\Programs\Python\Python312\Lib\site-packages\pydantic\_internal\_config.py:341: UserWarning: Valid config keys have changed in V2:
* 'orm mode' has been renamed to 'from_attributes'
  warnings.warn(message, UserWarning)
INFO: Started server process [19360]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Paso 4: Probar el servicio de usuario



Paso 5: Implementación del servicio de tareas

database.py : Crea la clase de base de datos y se guardará como **task-service/app/database.py** y coloca el siguiente código.

```
from sqlalchemy import create_engine
from sqlalchemy . ext . declarative import declarative_base
from sqlalchemy.orm import sessionmaker
```

```
SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"

motor = crear_motor ( SQLALCHEMY_DATABASE_URL )
SessionLocal = sessionmaker ( autocommit = False , autoflush
= False , bind = engine )
Base = base_declarativa ()
```

```
def get_db ():
    db = SessionLocal ()
    intentar :
        rendimiento db
    finalmente :
        db.close ()
```

model.py : Crea la clase del modelo y se guardará como **task-service/app/model.py** y coloca el siguiente código.

```
from sqlalchemy import Column , Integer , String
desde .database importar Base
```

```
class Usuario ( Base ):
    __tablename__ = "usuarios"

    id = Column ( Entero , clave_primaria = Verdadero ,
índice = Verdadero )
    nombre de usuario = Column ( Cadena , único = Verdadero
, índice = Verdadero )
    correo electrónico = Column ( Cadena , único = Verdadero
, índice = Verdadero )
    nombre_completo = Column ( Cadena )
```

routes.py : Crea la clase routes y se guardará como **task-service/app/routes.py** y coloca el siguiente código.

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from . import models, schemas
from .database import get_db
```

```
router = APIRouter()
```

```
@router.post("/", response_model=schemas.User)
def create_user(user: schemas.UserCreate, db: Session =
Depends(get_db)):
    db_user = models.User(username=user.username,
email=user.email, full_name=user.full_name)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
```

```
@router.get("/{user_id}", response_model=schemas.User)
def read_user(user_id: int, db: Session = Depends(get_db)):
```

```

    db_user = db.query(models.User).filter(models.User.id ==
user_id).first()
    if db_user is None:
        raise HTTPException(status_code=404, detail="User not
found")
    return db_user

```

schema.py : Crea la clase de esquema y se guardará como **task-service/app/schema.py** y coloca el siguiente código.

```

from pydantic import BaseModel

```

```

class UserCreate ( BaseModel ):
    nombre de usuario : str
    correo electrónico : str
    nombre_completo : str

```

```

class Usuario ( BaseModel ):
    id : int
    nombre de usuario : str
    correo electrónico : str
    nombre_completo : str

```

```

Clase Configuración :
    orm_mode = Verdadero

```

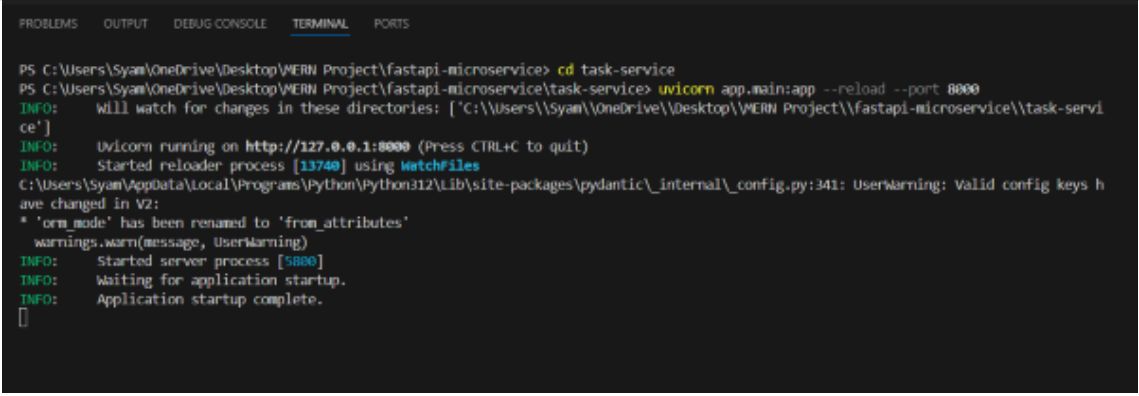
Paso 6: Ejecutar el servicio de tareas

Podemos utilizar lo siguiente para ejecutar el servicio de tareas.

```

uvicorn app.main:app --reload --port 8000

```

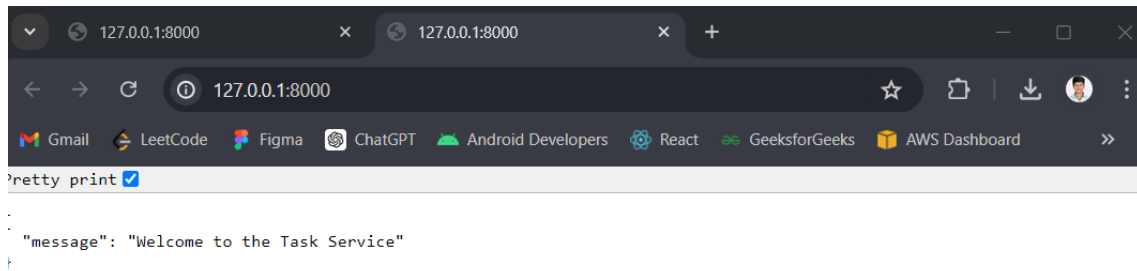


```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Syam\OneDrive\Desktop\WERN Project\fastapi-microservice> cd task-service
PS C:\Users\Syam\OneDrive\Desktop\WERN Project\fastapi-microservice\task-service> uvicorn app.main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\Users\Syam\OneDrive\Desktop\WERN Project\fastapi-microservice\task-servi
ce']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [13740] using WatchFiles
C:\Users\Syam\AppData\Local\Programs\Python\Python312\Lib\site-packages\pydantic\_internal\_config.py:341: UserWarning: Valid config keys h
ave changed in V2:
* 'orm_mode' has been renamed to 'from_attributes'
  warnings.warn(message, UserWarning)
INFO: Started server process [5000]
INFO: Waiting for application startup.
INFO: Application startup complete.
[]

```

Paso 7: Probar el servicio de tareas



cd user-service

uvicorn main:app --host 0.0.0.0 --port 8001 --reload

cd task-service

uvicorn main:app --host 0.0.0.0 --port 8002 --reload

FastAPI genera automáticamente una interfaz visual para probar tus endpoints sin escribir código adicional.

1. **Abre tu navegador** en las siguientes direcciones:
 - o **Usuarios:** <http://127.0.0.1:8001/docs>
 - o **Tareas:** <http://127.0.0.1:8002/docs>
2. Haz clic en el botón **"Try it out"** de cualquier método (POST o GET).
3. Modifica el cuerpo del JSON si es necesario y presiona **"Execute"**. Verás la respuesta en tiempo real (Código 200 OK).